

PHP静的解析現状確認会

ピクシブ株式会社 / [pixiv.git](https://github.com/pixiv)



pixiv Inc.

うさみけんた / **tadsan**

pixiv

はじめに

- 今回紹介するpixivの開発環境の改善は歴代の複数のチームメンバーの創意工夫の積み重ねであり、この資料は筆者(@tadsan)がその成果をまとめたものである
- PHPの型に関する見解は@tadsan個人の意見およびチーム内での議論に基づく

現状を三行で

- 未定義クラス・メソッド・定数・変数などの使用は簡単に検出できます
- PhpStorm、PhanやPHPStanなどのツールがありますが、それぞれの特徴が
- pixiv.gitの開発ではPull Requestごとに動的解析と静的解析の結果は概ね60秒程度で結果を表示できてます



tadsan

pixiv運営本部

自己紹介

- PHP以前にやってた言語はRuby, sh, Lisp
 - 経験として動的言語が中心
 - 静的型はF#, OCaml, Haskellに触れた程度
- 2012年11月にピクシブ入社 (最初はRuby)
- 2013年4月頃からPHPを書きはじめた

pixivのコードベースの特徴

- 手続き型に大きく寄った設計志向・アンチOOPの傾向が近い
 - クラスはオートロード可能な関数と定数の置き場としての扱い
 - PDOから取得した値をそのまま扱うことが多いので連想配列を多用
- 旧来の手法(≡ grep)による静的解析を容易にするためのコードベース
- 文字列・配列によるcallable表現はクロージャで表現する
- PSR-4を採用していない (useによる別名使用は極力しない)

pixivと静的解析

- (たぶん2014年頃) デプロイ時に必ず php -l でのチェックを実施
 - それ以前はSyntaxErrorを含むファイルをデプロイすることがまれにあった
- 2016年3月: 正規表現ベースのlintを導入
 - GitLabのMR単位でpushされる度に実行・コメント
- 2016年頃からPhan検証→ Phan静的解析がもたらす大PHP型検査時代(phpcon2016)
- 2018年6月: PHPStanを本格的にCIに導入

OOPに寄らない利点

- この利点はPhpStormとPHPStanによる静的解析によっておよそ無効化された。よって軽々に他者に推奨する戦略ではない。
- インスタンス化 → メソッド呼び出しの過程を経ると、grep/sed/awkなどのプリミティブなテキスト編集ツールだけではメソッド利用箇所の洗い出しやリファクタリングの妨げとなる。この状況は既に大量のコードがあるプロジェクトを効率よくメンテする側面においては大変に痛かった。
- 繰り返すが、現在においては型と静的解析の力を利用しながらPhpStormのリファクタリング機能などを利用し、過不足にほかのツールを利用するの懸命だろう

正規表現ベースのチェッカー(linter)

- 適用するパターン・除外するパターンを書ける
- MRではmasterとのdiffのあったファイルに対してチェッカーを適用
- 推奨されないバッドノウハウや表記揺れなどを抑止する目的
- 除外パターンも書けるので特定の場合のみ例外的に許可もしやすい
 - 例: テストコードでのみ使用可
- パターンのテストもあるのでチームメンバーが必要に応じて加除


```

'@pixiv-lib/.*\php$@' => [
  [
    'level' => 'error',
    'desc'  => 'pixiv-libで定義されているクラスは include_once しないこと',
    'pattern' => [
      '/include_once[ \(\] *INC_PATH /' => false,
      '@/stacc/api\php@' => true,
    ],
    'tests' => [
      ['pixiv-lib/foo.php', "include_once INC_PATH . '/Awesome/Class.php';",
true],
      ['pixiv-lib/foo.php', "include_once INC_PATH . '/stacc/api.php';",
false],
      ['pixiv-lib/foo.php', "include_once INC_PATH . 'HogeClass.php';", true],
      ['foo.php', "include_once INC_PATH . '/Awesome/Class.php';", false],
      ['foo.php', "include_once INC_PATH . '/stacc/api.php';", false],
      ['foo.php', "include_once INC_PATH . 'HogeClass.php';", false],
    ],
  ],
],

```

Phan

- コードを強力に静的解析する
- 連想配列の要素や特定のリテラルが来ること期待して解析できる
 - `@phan-return array{mode:'foo'|'bar',hoge:string}`
 - 明示しなくても実行パスを収集して最大限に型を推論してくれる
- 解析の度に全コードをスキャンするので大規模なコードベースでは、とても時間がかかる

PHPStan

- コードを強力に解析するが、静的情報だけではなく実行時情報を活用する
- 正確に検査するためには解析対象をオートロード(あるいは事前読み込み)できるようにすることが前提
- 一回のプロセス起動で多数のファイルを同時に解析すると多量のメモリを消費するので、Phanと違ってプロジェクトの全クラスを一度に検査する用途は向かない

pixivのCI

- PHPUnit(およびJSのテスト)による動的解析と
PHPStanと正規表現ベースのlinterによる静的解析の二本立て
- 前述の通り、静的解析にはPhanではなくPHPStanを採用している
- CI実行環境の改善により、GitLabのMerge Request(GitHubにおけるPull Request)のブランチにpushしてから60秒前後で動的解析と静的解析の結果が得られる

PHPと型

PHPアプリケーションと型

- そもそもが動的型付け言語で、静的に型を付ける上で不利な面も多い
- `$_GET`, `$_POST`, `PDO(ATTR_EMULATE_PREPARES時)`などは、あらゆる値を文字列型(または再帰的に文字列を要素とする配列)として扱う
 - そのような値に対して「適切なキャスト」を書くのは「とても」難しい
 - 不用意な判定で`string->int`にキャストするのはバグや脆弱性の温床になる
 - PHP7の型宣言はデフォルトでキャストするので強いリスクがある
(`declare(strict_types=1);`が必須)

型宣言への態度

- 既存の大規模なプロジェクトへの適用には強い懸念がある
- 特にstring->intのキャストが発生するケース
 - `declare(strict_types=1)`設定済みならPHP処理系が、
未設定なら人間が不用意にキャストしてしまうことがある
- 新規のプロジェクトでも、値を適切に扱える環境で慎重に導入したい
- pixivではスカラー型宣言(int, string, bool)は利用せずPHPDocに留めたい

ちょっと気になってるけど使っていない

- <https://github.com/sensiolabs-de/deptrac>
- <https://github.com/vimeo/psalm>